

SQL Normalization Forms

Normalization is the process of organizing data in a database to:

- Reduce data redundancy
 - Improve data integrity
 - Avoid update/insert/delete anomalies
-

Types of Normal Forms in SQL

1. First Normal Form (1NF)
 2. Second Normal Form (2NF)
 3. Third Normal Form (3NF)
-

1. First Normal Form (1NF)

Rule

A table is in 1NF if:

- Each column contains atomic (single) values
 - No repeating groups or arrays
 - Each row is unique
-

Example (Not in 1NF)

StudentID	Name	Subjects
1	Raj	Math, Science
2	Anu	English, Physics

Problem:

- Multiple values stored in one column
-

Convert to 1NF

StudentID	Name	Subject
1	Raj	Math
1	Raj	Science
2	Anu	English
2	Anu	Physics

Now:

- One value per cell
 - Atomic data
-

SQL Example

```
CREATE TABLE StudentSubjects (  
  StudentID INT,  
  Name VARCHAR(50),  
  Subject VARCHAR(50)  
);
```

2. Second Normal Form (2NF)

Rule

A table is in 2NF if:

- It is already in 1NF
- No partial dependency exists

Partial dependency means:

A non-key column depends on only part of a composite primary key.

Example (Not in 2NF)

StudentID	CourseID	StudentName	CourseName
1	C101	Raj	SQL
1	C102	Raj	Python

Composite Key:

(StudentID, CourseID)

Problem:

- StudentName depends only on StudentID
 - CourseName depends only on CourseID
-

Convert to 2NF

Students Table

StudentID	StudentName
1	Raj

Courses Table

CourseID	CourseName
C101	SQL
C102	Python

Enrollments Table

StudentID	CourseID
1	C101
1	C102

SQL Example

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50)  
);
```

```
CREATE TABLE Courses (  
    CourseID VARCHAR(10) PRIMARY KEY,  
    CourseName VARCHAR(50)  
);
```

```
CREATE TABLE Enrollments (  
    StudentID INT,  
    CourseID VARCHAR(10),  
    PRIMARY KEY (StudentID, CourseID)  
);
```

3. Third Normal Form (3NF)

Rule

A table is in 3NF if:

- It is already in 2NF
- No transitive dependency exists

Transitive dependency:

Non-key column depends on another non-key column.

Example (Not in 3NF)

EmpID	EmpName	DeptID	DeptName
1	Raj	D1	HR
2	Anu	D2	IT

Problem:

- DeptName depends on DeptID
 - DeptID depends on EmpID indirectly
-

Convert to 3NF

Employees Table

EmpID	EmpName	DeptID
1	Raj	D1

Departments Table

DeptID	DeptName
D1	HR

SQL Example

```
CREATE TABLE Departments (  
  DeptID VARCHAR(10) PRIMARY KEY,  
  DeptName VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
  EmpID INT PRIMARY KEY,  
  EmpName VARCHAR(50),  
  DeptID VARCHAR(10),  
  FOREIGN KEY (DeptID)  
    REFERENCES Departments(DeptID)  
);
```

Quick Summary Table

Normal Form	Removes
1NF	Repeating groups
2NF	Partial dependency
3NF	Transitive dependency

Real-World Example

Imagine an E-commerce Database:

- 1NF → Separate multiple phone numbers
 - 2NF → Separate customer and order details
 - 3NF → Separate city and postal code tables
-

Easy Memory Trick

Form	Remember
1NF	Atomic values
2NF	Full dependency
3NF	No indirect dependency

Important Interview Questions

1. Difference between 2NF and 3NF?
 2. What is transitive dependency?
 3. Difference between 3NF and BCNF?
 4. Advantages and disadvantages of normalization?
 5. When should denormalization be used?
 6. Explain normalization using a real-time project.
-

Advantages of Normalization

- Reduces redundancy
 - Saves storage
 - Improves consistency
 - Easier maintenance
 - Better data integrity
-

Disadvantages

- More joins
 - Complex queries
 - Performance overhead in large analytics systems
-

When Denormalization is Used

Used in:

- Data Warehouses
- OLAP systems
- Reporting databases
- High-performance read systems

Example:

Star Schema in Data Warehousing often uses denormalization.